CHAKRAVYUH

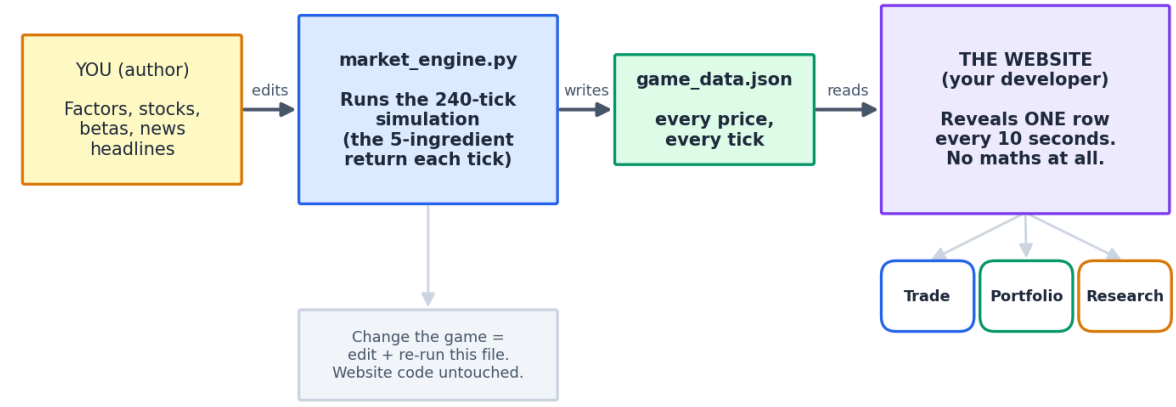
Trading-Game Engine & Website

Full Developer Handover — Backend Model (v5) + Frontend Design

The maths, the code, the algorithm, and how the website plugs into it.

How the whole system fits together

The maths runs ONCE before the game. The website just plays back a file.



Version: v5 (240 ticks · two 20-min sessions · one tick / 10 seconds)

Audience: website developer (code + integration) and model owner (maths).

How to read this document

This one document contains everything needed to build the game. It is written for two readers, and each part is labelled so you can jump to what you need:

- **The developer (you, building the website) —** read Part 1, Part 2, Part 5, and Part 6. You never have to touch or understand the maths. Your job is to play back a data file and draw three tabs.
- **The model owner —** read Part 3 and Part 4 for the full intuition and the exact equations, and Part 7 to tune the feel.

Table of contents

Right-click here and choose “Update Field” to build the contents.

The single most important idea

The maths engine runs ONCE, before the game starts, and writes the entire game to one file (game_data.json) — every stock's price at every 10-second tick. The website does NOT calculate anything. It just reads that file and reveals one row every 10 seconds, like a slideshow. This completely separates the developer's job from the maths.

Part 1 — The big picture

What the game is

Players are given a starting amount of cash and a universe of 10 Indian stocks. Over two 20-minute sessions, news headlines drop in roughly every minute. Each headline moves the market in a way that depends on which stocks are exposed to it. Players read the news, read each stock's background research, and buy or sell to grow their portfolio while respecting a drawdown limit. The skill being taught is: match the catalyst (news) to the exposed stock, size the trade, and manage risk.

The three moving parts

- **The engine (market_engine.py) —** a self-contained Python program. It holds the whole market model. You (the owner) edit the stocks, factors, and headlines; you never touch the website.
- **The data file (game_data.json) —** the engine's output. A plain list of every price at every tick, plus the news that fires and the research universe. This is the ONLY thing the website reads.
- **The website (three tabs) —** Trade, Portfolio, Research. Pure front-end. It plays the data file back on a 10-second timer and lets players trade against it.

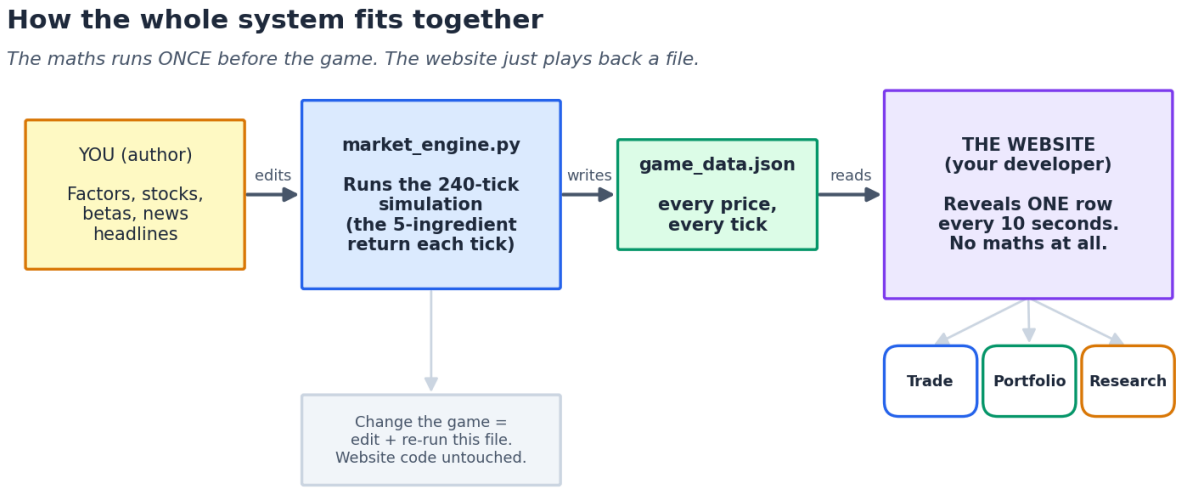


Figure. The maths is pre-computed once into a file; the website is a timed slideshow over that file.

Why pre-generate instead of computing live?

It is fair (every player sees the exact same market during a given game), cheap (zero maths on the server during play), and it lets the owner inspect and approve a game before it goes live. The developer's whole task becomes 'read row t, show it'.

Every game is new — no two sessions are ever the same

By default the engine draws a FRESH random seed on every run, so each generated game is completely different. And within a single game the two 20-minute sessions already differ from each other (the news is re-shuffled and re-drawn per session). You never reuse a game unless you choose to. The one seed is saved inside the file purely so a specific game CAN be reproduced on demand (to approve it before an event, or to settle a dispute) — reproducibility is an option you reach for, not the default.

Part 2 — Frontend design (for the developer)

Three tabs, taken directly from the wireframes. Below each is exactly what it shows and which fields of game_data.json feed it. Clean recreations are shown; the layout and every widget match the sketches.

2.1 Trade tab

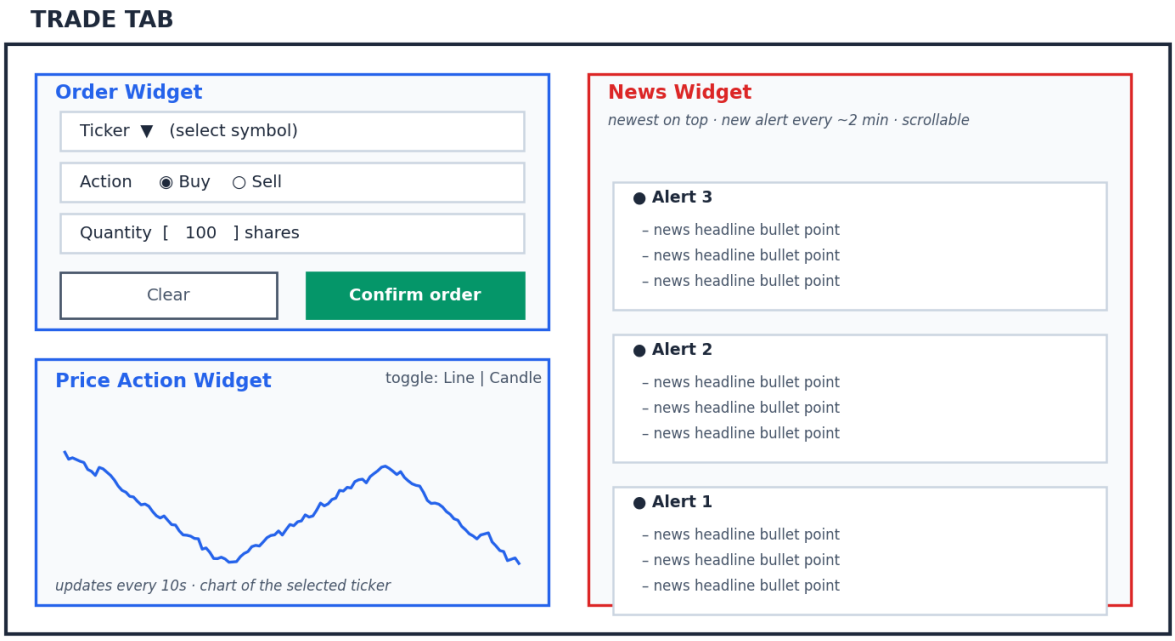


Figure. Trade tab — Order widget (top-left), Price Action chart (bottom-left), News feed (right).

Order widget

- **Ticker dropdown —** select the symbol to trade (list comes from meta.tickers in the JSON).
- **Action —** Buy or Sell.
- **Quantity —** number of shares.
- **Clear button —** resets the form.
- **Confirm order button —** executes the trade at the currently-shown price of the selected ticker.

Price Action widget

- Shows the price chart of the ticker currently selected in the Order widget.
- **Updates every 10 seconds —** one new price point per tick, straight from the JSON.
- **Two chart types via a toggle —** (i) simple line chart, (ii) candlestick. Ship the line chart first; candles need OHLC, which you build by grouping several ticks into one candle (see Part 6).

News widget

- **Live notifications —** a new alert appears roughly every 2 minutes (display cadence; see the note below).
- **Each notification has 3–5 bullet-point headlines —** these come from the news array on each tick row.
- **Newest on top —** new alerts push older ones down; the whole feed is scrollable.

Cadence note for the developer

The engine fires one news event per in-game minute (20 per session). Your wireframe shows a notification every ~2 minutes with 3–5 bullets. Simplest reconciliation: collect the headlines that fired since the last notification and show them as one bundled alert card on your chosen display cadence. The bullets are already in each tick's news list — you are only choosing how often to surface them.

Security: never reveal the hidden fields

Each headline in the JSON carries factor, ticker and score fields. These are the 'answer key' — the WHY players are meant to deduce. Show ONLY the bullet text to players. Never render factor/ticker/score.

2.2 Portfolio tab

Figure. Portfolio tab — six metric cards over a portfolio-vs-market equity chart.

- **Available Cash —** starting cash minus buys plus sells.
- **Unrealised Gain —** for open positions: shares × (current price − average cost).
- **Total Portfolio —** cash + current market value of all holdings.

- **Absolute Return % —** (total portfolio − starting capital) / starting capital.
- **Sharpe Ratio —** average of the portfolio's per-tick returns ÷ their standard deviation (a reward-per-unit-risk score).
- **vs. Market Return % —** portfolio return minus the equal-weight market return over the same period.
- **Equity Value widget —** two lines over time. Black = the player's portfolio value; Red = the market (equal-weight index of all tickers). Formulae for all of these are in Part 6.

2.3 Research tab

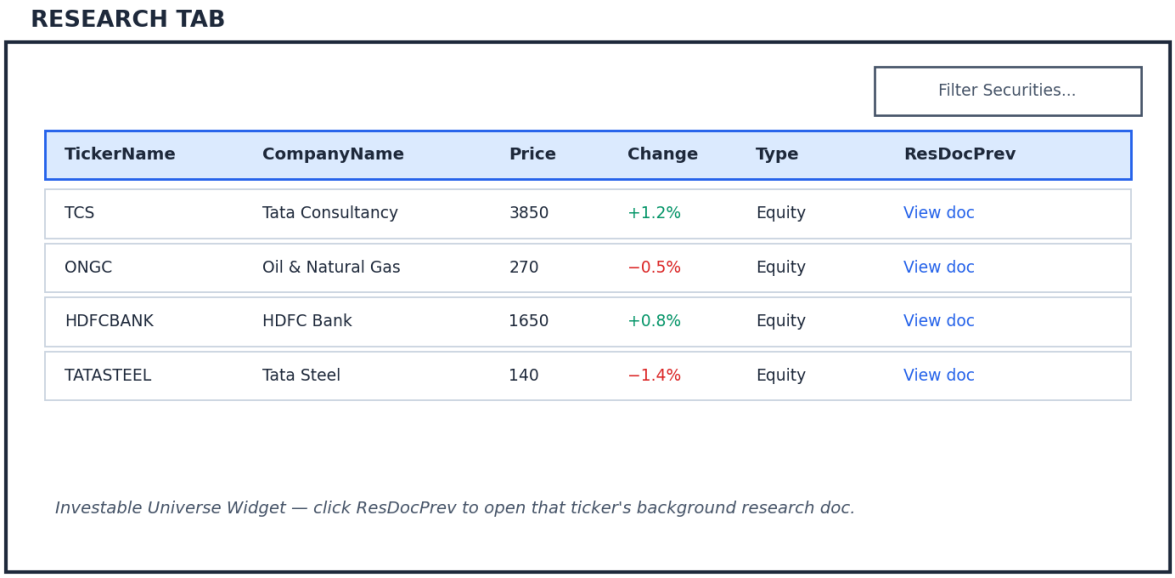


Figure. Research tab — the investable-universe table with a per-ticker research document.

- **A table (dataframe) of the investable universe —** columns: TickerName, CompanyName, Price, Change, Type, ResDocPrev.
- **Filter Securities —** a search/filter box over the table.
- **ResDocPrev —** a link in each row. Clicking it opens that ticker's background research document (the research field in the JSON's universe list). This is where players learn a stock's exposures.

Part 3 — The model, in plain English

This part explains how the engine thinks, with no heavy maths. Part 4 gives the exact equations. If you only read one part to 'get it', read this one.

3.1 A price is just returns, stacked up

We never invent a price directly. We compute how much a stock MOVES this tick (its 'return'), then grow the previous price by it. Growing by e^(return) instead of adding a percentage means the price can wander anywhere but can never fall below zero.

```
price now  =  price a moment ago  ×  e^(this tick's return)
```

3.2 A return is the sum of five simple pieces

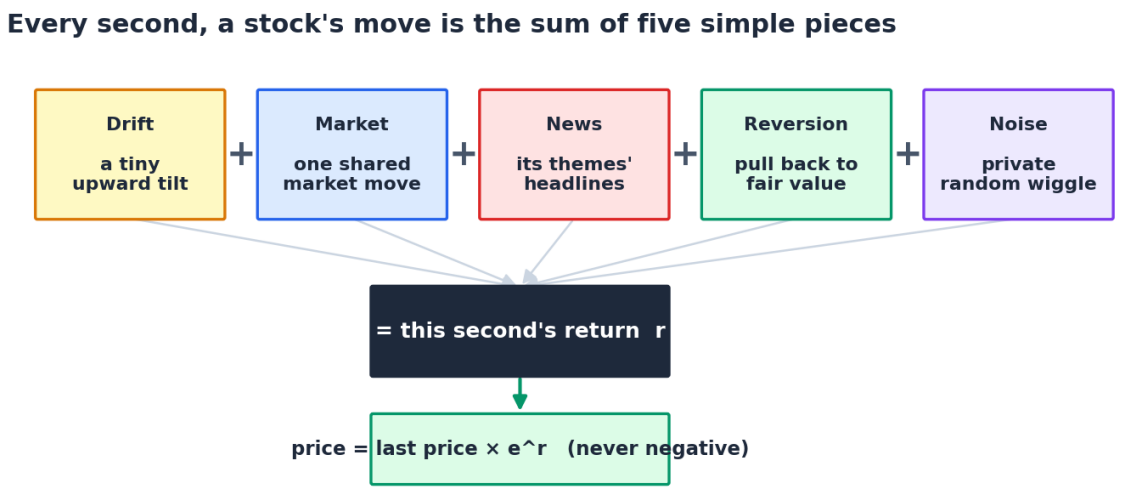


Figure. Every tick, each stock's move is drift + market + news + reversion + noise.

- **Drift —** a tiny constant upward tilt (markets gently rise over time).
- **Market —** one shared move that nudges every stock the same tick (some catch more of it than others).
- **News —** the interesting piece: how this stock's news themes moved (next section).
- **Reversion —** a spring that pulls a stock back toward its 'fair value' so shocks don't run forever.
- **Noise —** each stock's own private random wiggle.

3.3 How one headline becomes price action

A headline never touches a stock directly. It moves a THEME (a 'factor' like CrudeOil), and each stock reacts to that theme through a sensitivity number called a beta. Write one headline, tag it to one factor, and the whole board reacts correctly and automatically.

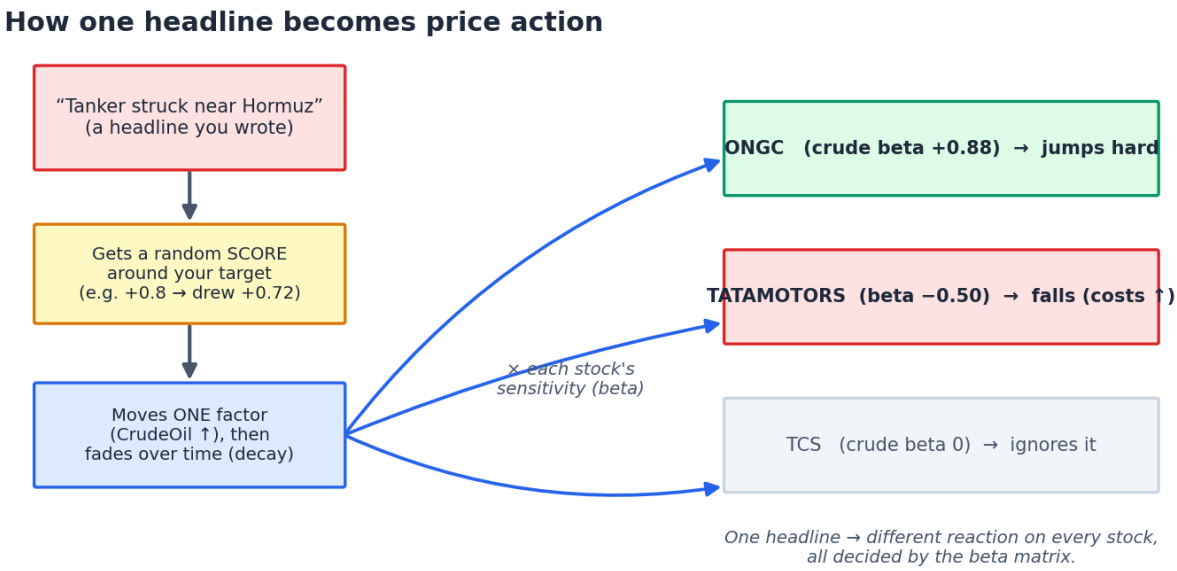


Figure. One headline → one factor → every stock reacts by its own beta (positive, negative, or zero).

- **The score —** each headline is given a target strength in −1…+1, then a slightly random draw around it, so the same headline never lands identically twice.
- **The decay —** a headline hits hardest the instant it fires, then fades. A big, slow story lingers; a blip is gone fast.

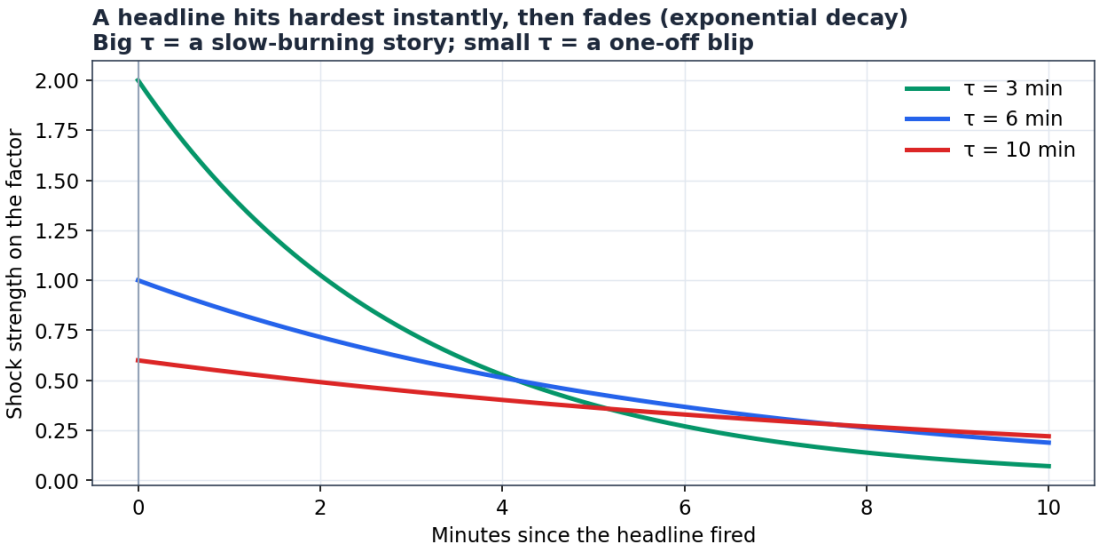


Figure. News fades exponentially. τ (tau) sets how long a story lingers.

3.4 Why prices don't run away forever (mean-reversion)

Early versions had a flaw: a big shock permanently relocated a price, so 'short it and hold' was free money. v5 fixes this. Each stock has a hidden fair-value anchor. Only 35% of any shock is

'real' and permanently moves the anchor; the other 65% is an overshoot that springs back. So genuinely bad news still leaves the stock lower, but the panic retraces — and you can't just hold the obvious direction forever.

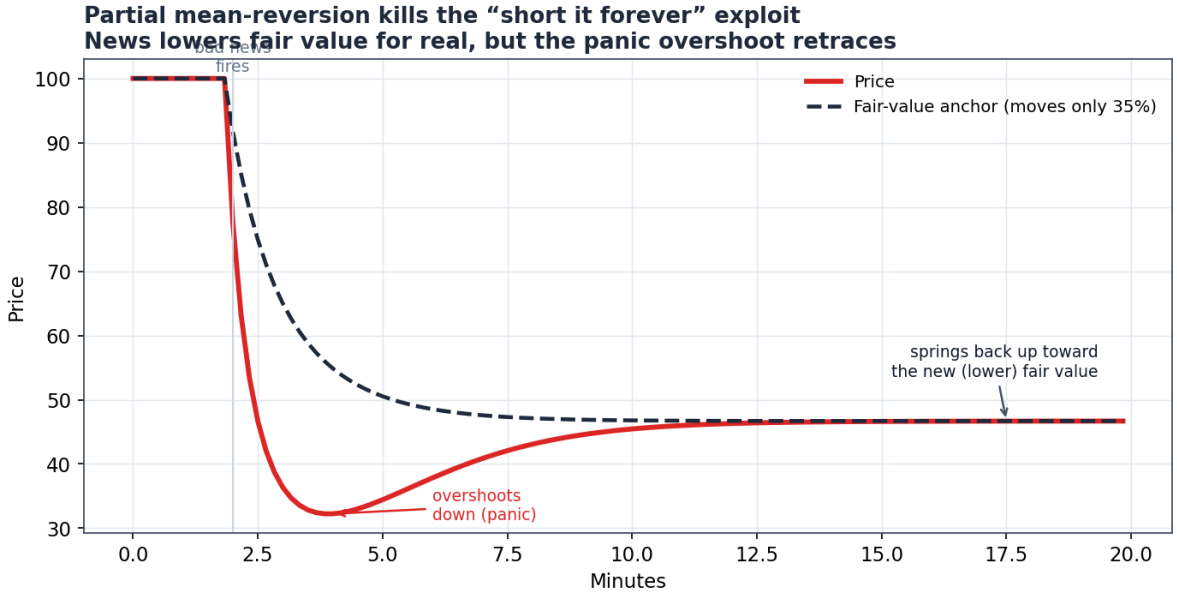


Figure. Price overshoots on panic, then springs back toward the new (lower) fair value.

3.5 Two touches that make it feel like a real market

- **Correlated themes —** the 10 factors don't move independently; related ones (e.g. Metals & CrudeOil) tend to move together. This is done once with a matrix 'square-root' trick (Cholesky).
- **Shuffled news order —** the headlines are dealt into different minutes each game (kept spaced out, with a few storylines pinned), so the same market plays as a different path every session.

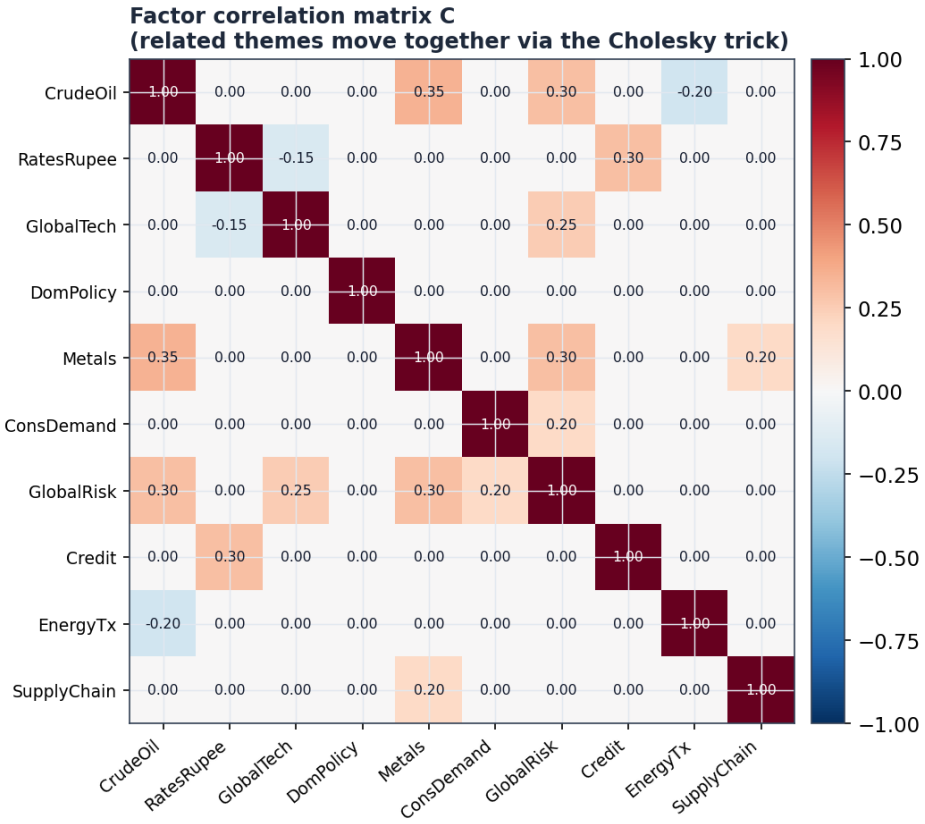


Figure. The factor correlation matrix — related themes move together.

3.6 What is random vs fixed (why no two games are alike)

When it is drawn	What is drawn
Once per game (frozen after)	Each stock's factor betas & market beta; each headline's exact strength (gain) & duration (tau); the news order.
Every tick	The shared market move; each factor's background wiggle; each stock's private noise.
When a headline fires	That headline's realised score (around its target).
Never (fixed config)	The 10 factors, the correlation matrix, drift, the reversion spring strengths.

3.7 The whole model, running for real

Here is the reconstructed v5 engine's actual output — one full game. Notice every stock bends only at the news it is exposed to, paths cross and reverse (so timing matters), and troughs bounce back (reversion working).

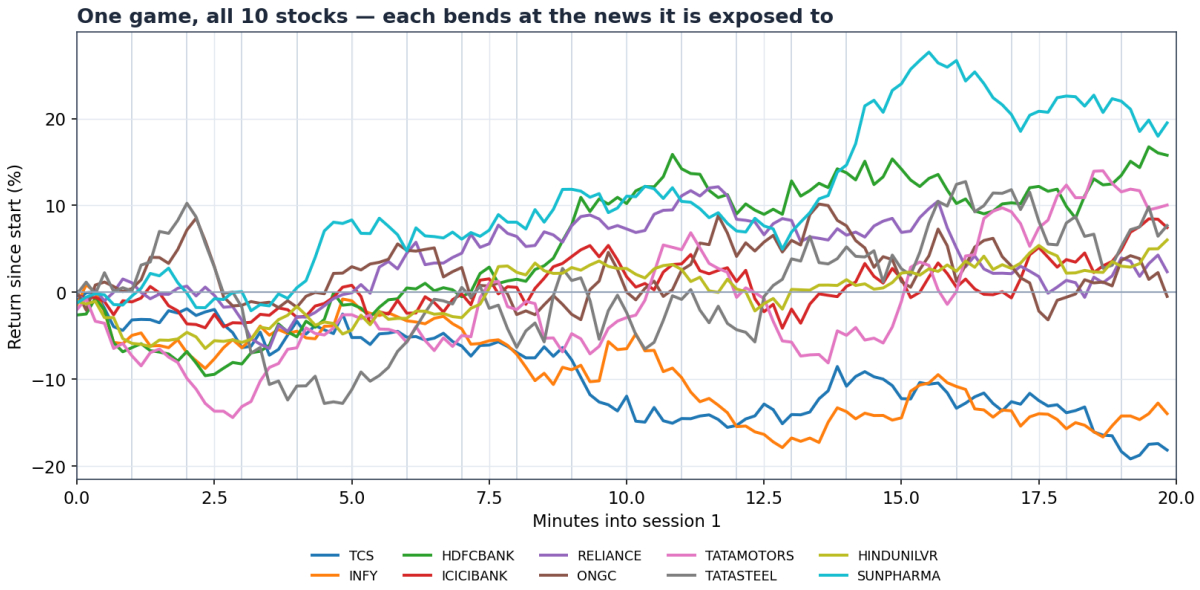


Figure. One game, all 10 stocks, session 1. Each line is driven by its own betas.

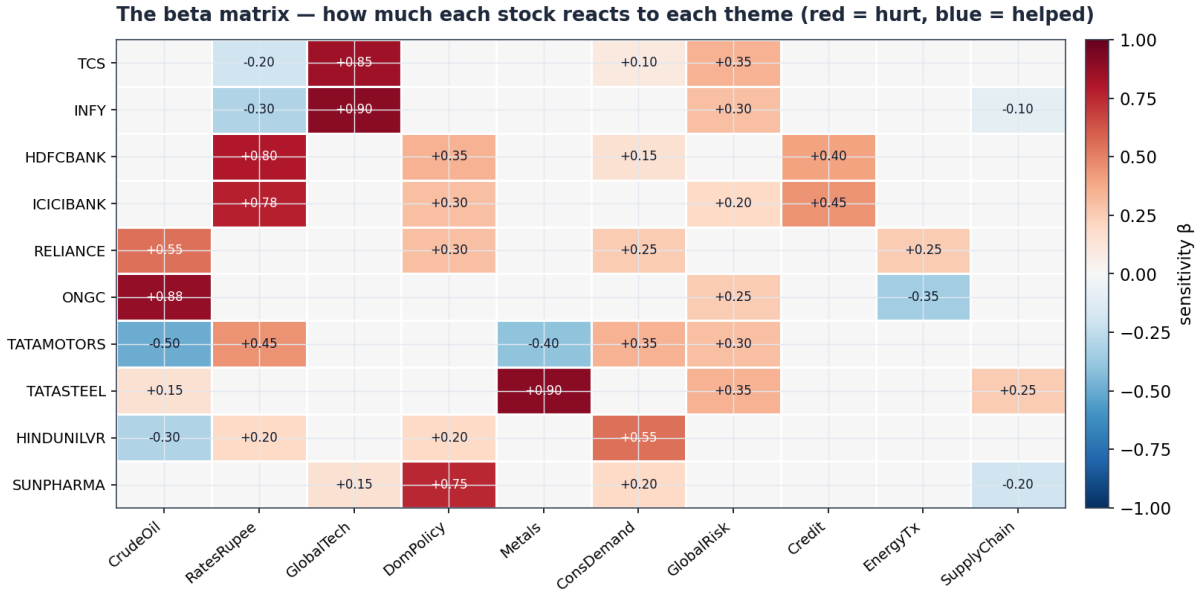


Figure. The beta matrix that produced those paths: how strongly each stock reacts to each theme.

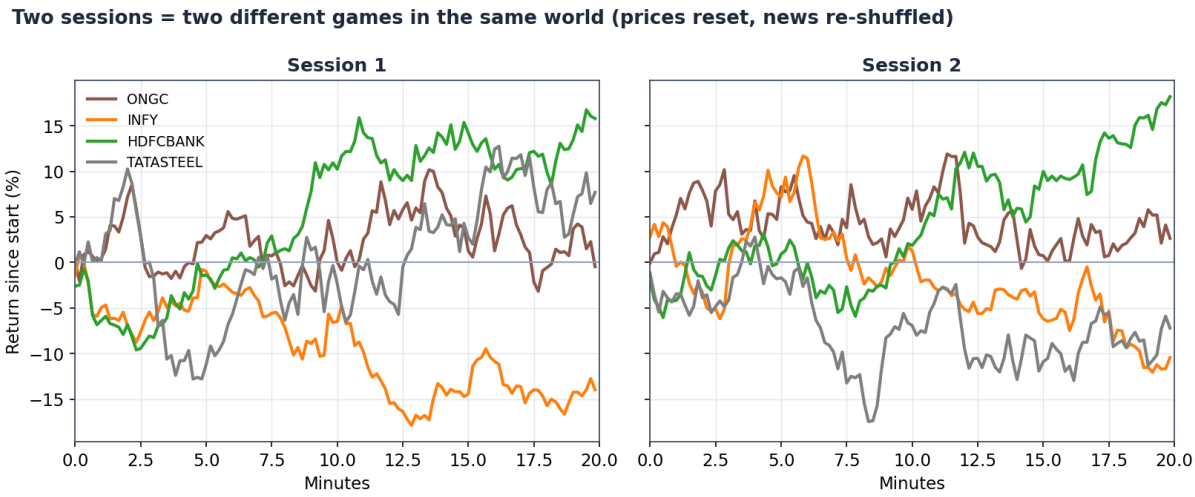


Figure. Two sessions are two different games in the same world — prices reset, news re-shuffled.

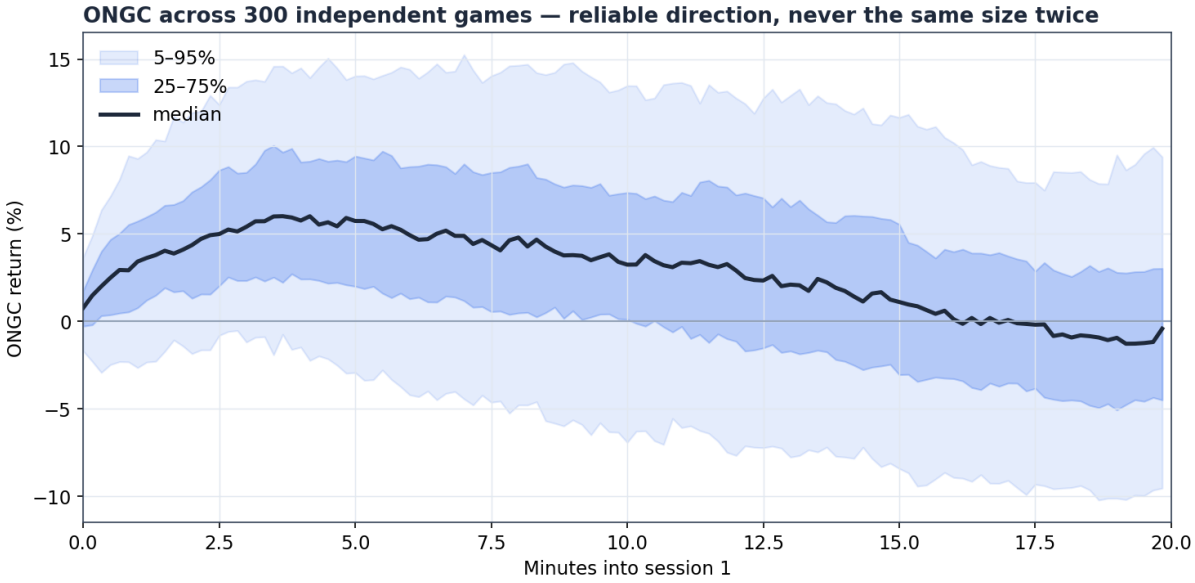


Figure. ONGC across 300 games: a reliable direction on its crude event, but never the same size twice.

Part 4 — The full maths (v5), for the model owner

Everything below is the exact specification, unwound from the price backward (as requested): first how price is built, then the return equation, then each term inside it. Notation: i = a stock, k = a factor, t = a tick (one 10-second step, t = 1…240), e = a news event.

4.1 Price recursion (how a path is built)

```
P[i,t]  =  P[i,t-1] · e^( r[i,t] )        (run this every tick)

P[i,t]  =  P[i,0] · exp( Σ_{s=1..t} r[i,s] )   (same thing, unrolled)
```

The left form is what the code runs — one multiply per tick. The right form shows the price is the start price times e raised to the running total of all returns so far (that inner sum is the random walk). Because e^(anything) > 0, price can never go negative.

4.2 The return equation (the core)

```
r[i,t] = μ[i] + β_mkt[i]·M[t] + N[i,t] + R[i,t] +
                v·m_idio·σ[i]·ε[i,t]
```

- **μ[i] drift —** a fixed per-tick constant (≈ 6.2×10^{-5}). Over 240 ticks it compounds to the baseline trend.
- **β_mkt[i]·M[t] market —** M[t] is one market-wide shock shared by all stocks this tick; β_mkt scales it per stock.
- **N[i,t] news —** factor exposure + any direct ticker headline (Section 4.4).
- **R[i,t] reversion —** the pull toward fair value (Section 4.6).
- **v·m_idio·σ[i]·ε[i,t] noise —** ε ~ N(0,1) fresh each tick; σ[i] is the stock's private volatility.

```
M[t] = v·σ_mkt·ξ[t],   ξ[t] ~ N(0,1)
```

4.3 Factor returns F[k,t] (the shared drivers)

```
F[t] = v·(L·z[t]) ⊙ σ_F   +   S[t]        (vector over the 10 factors)

F[k,t] = v·σ_F[k]·(L·z[t])[k]  +  S[k,t]   (one factor)
```

- **z[t] ~ N(0, I) —** 10 independent standard normals.
- **L —** the Cholesky factor of the correlation matrix C, i.e. $L\cdot L^{T} = C$. Multiplying independent noise by L turns it into correlated noise (so e.g. Metals and CrudeOil co-move).
- **σ_F —** each factor's background volatility; ⊙ is elementwise; v rescales for tick resolution.
- **S[k,t] —** the total news shock on factor k (next section).

4.4 The news term N[i,t]

$$N[i,t] = \beta[i]^{T} \cdot F[t] \; + \; S\_tk[i,t]$$
$$= \Sigma_{k=1..10} \; \beta[i,k] \cdot F[k,t] \; + \; S\_tk[i,t]$$

- **β[i]** — stock i's beta vector; each β[i,k] ∈ [−1,1], sign meaningful (an airline vs oil is negative). β[i,k]=0 means stock i ignores factor k entirely.
- **S_tk[i,t]** — a shock from any headline aimed directly at stock i (e.g. 'INFY cuts guidance'), same decay shape.

4.5 News shocks and the decay factor

A single event e fires at tick t_0 on either a factor or a ticker, and contributes at every later tick $t \geq t_0$:

$$shock\_e(t) = s\_e \cdot g\_e \cdot e^{-(t - t_0)/\tau\_e} \cdot \left(\tau\_ref / \tau\_e \right)$$

- **s_e score** — direction & intensity, drawn once when it fires: s_e = clip(N(s*_e, σ_s^2), −1, 1). Wide σ_s ⇒ events sometimes land weak or flip sign.
- **g_e gain** — peak magnitude (how big a deal the headline is).
- **e^(−(t−t_0)/τ_e) decay** — 1 at the instant it fires; ≈0.37 after τ_e ticks; ≈0 after 3τ_e.
- **τ_ref/τ_e normaliser** — keeps a shock's TOTAL integrated impact fixed no matter how long it lingers (∫ g·e^(−a/τ) da = g·τ, so dividing by τ and referencing τ_ref makes total ≈ g·τ_ref). τ_ref = 36.

$$S[k,t] = \Sigma_{\text{events on factor } k} \; shock\_e(t)$$
$$S\_tk[i,t] = \Sigma_{\text{events on ticker } i} \; shock\_e(t)$$

g_e and τ_e are themselves jittered once per game: g_e ~ N(g*, $(0.10 \cdot g^*)^2$), τ_e ~ N(τ*, $(0.10 \cdot \tau^*)^2$).

4.6 Mean-reversion and the moving anchor

Each stock carries a hidden fair-value anchor A[i,t]. The reversion term pulls the (log) price toward it, and the PERMANENT slice of each tick's news moves the anchor itself:

$$R[i,t] = \kappa[i] \cdot \left(\ln A[i,t] - \ln P[i,t-1] \right)$$
$$A[i,t] = A[i,t-1] \cdot \exp\left(\phi \cdot N[i,t] \right)$$

- **κ[i] spring strength** — how fast price snaps back (v5: ≈ 0.08). Bigger = faster retrace.
- **ϕ = 0.35 permanent fraction** — 35% of a news move permanently relocates fair value; the other 65% is a transient overshoot the spring pulls back. This is what kills 'short the shock forever'.

4.7 Correlation via Cholesky

$$C = L \cdot L^{T} \quad (C = \text{factor correlation matrix, } L \text{ lower-triangular})$$
$$z[t] \sim N(0, I), \quad \text{then} \quad \eta[t] = L \cdot z[t] \text{ has correlation } C$$

A hand-written correlation matrix can occasionally be invalid (non-positive-semi-definite) and Cholesky will refuse; keep it to sensible values, or repair to the nearest valid matrix. Convex blends of valid matrices stay valid.

4.8 The full per-tick algorithm

This is literally the body of the engine's step() function, in order, for each tick t (t_local within a session):

```
(a) FIRE due events:   for each event e with t0 == t:
        s_e = clip( Normal(target_e, SCORE_SPREAD), -1, +1 )

(b) SUM active shocks:
        S[k]   = Σ  s_e·g_e·exp(-(t-t0)/tau_e)·(REF_TAU/tau_e)   over factor
events
        S_tk[i]= Σ  (same)                                       over ticker
events

(c) FACTORS:   z ~ Normal(0, I_10)
        F = VOL_SCALE·(L @ z)·FACTOR_VOL + S

(d) MARKET:    M = VOL_SCALE·MKT_VOL·Normal(0,1)

(e) PER STOCK i:
        N      = beta[i]·F + S_tk[i]
        A[i]   = A[i]·exp(PERM_FRAC · N)              # move fair value
        R      = KAPPA·(ln A[i] - ln P[i])            # spring toward it
        idio   = VOL_SCALE·IDIO_MULT·sigma[i]·Normal(0,1)
        r      = DRIFT + mkt_beta[i]·M + N + R + idio # the 5 ingredients
        P[i]   = P[i]·exp(r)                          # grow price
```

4.9 Symbol table

Symbol	Meaning	Drawn / fixed
P[i,t]	price of stock i at tick t	state
r[i,t]	log-return of i at t	computed
μ[i]	drift	fixed (≈6.2e-5)
M[t], ξ[t]	market shock / its unit normal	per tick
β_mkt[i]	market beta	per game, N(1, 0.05^2)
F[k,t]	factor return	per tick
z[t]	10 independent normals	per tick
L, C	Cholesky factor / correlation matrix	fixed ($L \cdot L^T = C$)
σ_F	factor background vols	fixed
β[i,k]	stock i's factor betas	per game, N(target, 0.05^2), sign-kept

s_e, s*_e	event score / its target	score per fire; target fixed
g_e, τ_e	event gain / decay length	per game (jittered)
τ_ref	reference decay (normaliser)	fixed = 36
S_tk[i,t]	direct ticker-news shock	per tick
A[i,t]	fair-value anchor	state
κ[i]	reversion speed	fixed per stock (≈0.08)
ϕ	permanent fraction of news	fixed = 0.35
σ[i], ε[i,t]	idio vol / its unit normal	vol fixed, ε per tick
ν, m_idio, σ_s	vol-scale / idio-mult / score-spread	fixed

4.10 v5 constants (re-derived for the 240-tick, 10-second clock)

Everything scales off the tick clock. Three rules generate the numbers: noise scales with √(ticks), drift scales with 1/ticks, and shock decay is authored in real minutes then converted to ticks.

Constant	Value	Why
ticks/session · total	120 · 240	20 min × 6 ticks/min, two sessions
SUBTICKS (ticks/min)	6	60 sec ÷ 10 sec
drift μ	6.2e-5	≈1.5% baseline trend ÷ 240 ticks
VOL_SCALE ν	≈ 1.0	at 10-sec ticks the fine-grid shrink factor ≈ 1
REF_TAU	36	≈ a 6-minute reference shock; keeps shock size stable
event tau	author in minutes × 6	a 6-min story = 36 ticks
κ (reversion)	≈ 0.08	bumped up so retrace completes within 120 ticks
ϕ (perm_frac)	0.35	unchanged — it is a ratio, resolution-independent
σ_s (score spread)	0.20	the core surprise dial
m_idio (idio mult)	1.6	private-noise loudness

Why ϕ (perm_frac) does NOT change with the clock

drift, noise and tau all reference the tick grid directly, so they rescale when the clock changes. ϕ is a fraction of a shock's total impact — and that total is already held fixed by the τ_ref normaliser. It splits the same total the same way regardless of resolution, so it stays 0.35.

Part 5 — The engine code

The whole model is one file, market_engine.py, organised into labelled sections. To change the game you edit Sections 3–5 (factors, stocks, news) and re-run. The website never changes.

5.1 File layout

Section	What it holds	Do you edit it?
1 Clock	tick timing (240 ticks, 10 sec)	rarely
2 Knobs	the v5 constants from Part 4.10	to tune feel (Part 7)
3 Factors	the 10 themes + correlation matrix	yes — to change themes
4 Stocks	start price, vols, beta matrix, company names	yes — to change the universe
5 News	the per-session headlines	yes — to write the story
6 Engine	MarketEngine class + step()	no (this is the model)
7 Export	build_game() → game_data.json	no

5.2 The contract: one class, one step

One engine = one game. Construct it once and keep it for the whole session. (Betas are drawn in the constructor; rebuilding it per tick would re-roll the 'world' every tick — never do that.) The entire surface area the rest of the system needs is a single method:

```
eng = MarketEngine(seed=7)      # build ONE per game, hold it
out = eng.step(session, t)      # advance exactly one tick
#  out['prices'] -> dict {ticker: price}
#  out['news']   -> list of headlines that fired this tick
```

5.3 The step() function (the model, line by line)

This is the real deliverable. Every line maps to an equation in Part 4.8. It is short on purpose.

```
def step(self, session, t_local):
    events = self.sessions[session]
    # (a) fire due events -> realise each one's random score
    fired_now = []
    for e in events:
        if e['fire_tick'] == t_local:
            raw = self.rng.normal(e['target'], SCORE_SPREAD)
            e['score'] = float(np.clip(raw, -1, 1))
            fired_now.append(e)
    # (b) sum every active shock, split into factor vs ticker
    factor_shock = np.zeros(NF); ticker_shock = np.zeros(NT)
    for e in events:
        if e['score'] is None: continue
        val = self._shock(e, t_local)          # score*gain*exp(-
age/tau)*(REF_TAU/tau)
        if e['factor']: factor_shock[FIDX[e['factor']]] += val
        if e['ticker']: ticker_shock[TICKERS.index(e['ticker'])] += val
    # (c) correlated factor background + news
```

```
    z = self.rng.standard_normal(NF)
    F = VOL_SCALE * (self.L @ z) * FACTOR_VOL + factor_shock
    # (d) one shared market move
    M = VOL_SCALE * MKT_VOL * self.rng.standard_normal()
    # (e) per stock: news -> anchor -> reversion -> return -> price
    N = self.beta @ F + ticker_shock
    self.anchor *= np.exp(PERM_FRAC * N)
    R = KAPPA * (np.log(self.anchor) - np.log(self.px))
    idio = VOL_SCALE * IDIO_MULT * self.idio_vol * self.rng.standard_normal(NT)
    r = DRIFT + self.mkt_beta * M + N + R + idio
    self.px *= np.exp(r)
    return dict(prices=self.px.copy(), factors=F, news=fired_now)
```

5.4 Authoring a stock and a headline

You never write maths — you fill in config. A stock is a start price, a volatility, and a dictionary of sensitivities. A headline is a minute, a target factor (or ticker), a strength, a duration, and the bullet text.

```
# a stock (Section 4):
'ONGC': dict(px=270, vol=0.009,
             betas={'CrudeOil': 0.88, 'EnergyTx': -0.35, 'GlobalRisk': 0.25}),

# a headline (Section 5):
E(minute=1, factor='CrudeOil', target=+0.80, gain=0.055, tau_min=6,
  bullets=['Tanker attacked near Hormuz', 'Crude spikes on supply fears'])
```

5.5 Producing the data file

```
python market_engine.py            # DEFAULT: fresh random seed -> a brand-new
game every run
python market_engine.py 7          # optional: pin seed 7 -> reproduce that exact
game
#  -> writes game_data.json, prints the seed it used, and a per-session return
summary
```

New game every run — guaranteed

With no argument the engine invents a new random seed each time, so no two generated games are alike, and the two sessions inside a game already differ from one another. You only pass a seed when you WANT to reproduce a specific game (to approve it before an event, or check a dispute). The seed used is always printed and saved into the file, so any game stays reproducible without ever having to reuse one. For a live event: generate once, review it, then serve that one approved file to all players.

5.6 The JSON the website reads

game_data.json has three parts: meta (setup), universe (the Research tab), and rows (one per tick).

```
{
  "meta": { "seed": 481923, "sessions": 2, "ticks_per_session": 120,   // seed
```

```
auto-generated & saved
            "tick_seconds": 10, "tickers": ["TCS", "INFY", ...],
            "start_prices": { "TCS": 3850, ... } },
  "universe": [ { "ticker": "ONGC", "company": "Oil & Natural Gas Corp",
                  "start_price": 270, "type": "Equity",
                  "research": "...background + sensitivities..." }, ... ],
  "rows": [
    { "session": 0, "tick": 0,
      "prices": { "TCS": 3850.0, "INFY": 1520.0, ... },
      "news": [ { "bullets": ["...", "..."],
                  "factor": "CrudeOil", "ticker": null, "score": 0.688 } ] },
    ...  (240 rows total)
  ]
}
```

Reminder

factor, ticker and score inside each news item are HIDDEN metadata for you — never show them to players. Render only bullets.

Part 6 — Wiring the website to the data (developer guide)

No maths here — just how to turn the JSON into the three tabs. Everything is a timed slideshow plus some simple bookkeeping. Snippets are plain JavaScript you can adapt to React/Vue/whatever you use.

6.1 The heartbeat: reveal one row every 10 seconds

```
const game = await fetch('game_data.json').then(r => r.json());
let t = 0;                                  // current tick index into rows
const rows = game.rows;

const timer = setInterval(() => {
  const row = rows[t];
  updatePriceChart(row);                    // Trade tab
  if (row.news.length) pushNews(row.news);  // News widget
  refreshPortfolio(row);                    // Portfolio tab
  t++;
  if (t >= rows.length) clearInterval(timer);
}, 10000);                                  // 10 seconds = one tick
```

That loop is the entire engine of the website. Everything else is drawing.

6.2 Order widget — buying and selling

Keep a little state: cash, and a map of holdings. On Confirm, execute at the selected ticker's current price.

```
let cash = 1_000_000;                       // starting capital
const holdings = {};                        // ticker -> { shares, avgCost }

function confirmOrder(ticker, side, qty) {
  const price = rows[t].prices[ticker];     // current shown price
  const cost  = price * qty;
  if (side === 'BUY') {
    if (cost > cash) return alert('Not enough cash');
    const h = holdings[ticker] || { shares: 0, avgCost: 0 };
    h.avgCost = (h.avgCost*h.shares + cost) / (h.shares + qty);
    h.shares += qty; holdings[ticker] = h; cash -= cost;
  } else {                                  // SELL
    const h = holdings[ticker]; if (!h || h.shares < qty) return alert('Not enough
shares');
    h.shares -= qty; cash += cost;
  }
}
```

6.3 Price Action widget — line now, candles later

The JSON gives one price per tick, which is exactly a line chart. For the candlestick toggle, group N ticks into one candle: open = first price in the group, close = last, high/low = max/min.

```
// line chart: just push the point for the selected ticker each tick
function updatePriceChart(row) {
  const tk = selectedTicker;
  linePoints.push({ x: row.tick, y: row.prices[tk] });
  chart.render(linePoints);
}

// candles: bucket every 6 ticks (= 1 minute) into one OHLC bar
function toCandles(prices, size = 6) {
  const candles = [];
  for (let i = 0; i < prices.length; i += size) {
    const slice = prices.slice(i, i + size);
    candles.push({ open: slice[0], close: slice[slice.length-1],
                   high: Math.max(...slice), low: Math.min(...slice) });
  }
  return candles;
}
```

6.4 News widget — newest on top, scrollable

```
function pushNews(items) {
  for (const n of items) {
    const card = document.createElement('div');
    card.className = 'news-card';
    card.innerHTML = n.bullets.map(b => `<li>${b}</li>`).join('');
    feed.prepend(card);           // newest on top; older ones slide down
    //  DO NOT render n.factor / n.ticker / n.score  (hidden answer key)
  }
}
```

6.5 Portfolio tab — the six metrics

All six cards are simple bookkeeping over the current prices. Compute them each tick inside refreshPortfolio.

```
function refreshPortfolio(row) {
  let positionValue = 0, unrealised = 0;
  for (const tk in holdings) {
    const h = holdings[tk], px = row.prices[tk];
    positionValue += h.shares * px;
    unrealised    += h.shares * (px - h.avgCost);
  }
  const total   = cash + positionValue;          // Total Portfolio
  const absRet  = (total - START_CAPITAL) / START_CAPITAL * 100;   // Absolute
Return %

  //  track portfolio value each tick to get returns for Sharpe & the chart
  equitySeries.push(total);
  const rets = pctChanges(equitySeries);         // tick-to-tick % changes
  const sharpe = mean(rets) / (std(rets) || 1);  // Sharpe Ratio (reward per unit
risk)

  //  market = equal-weight index of all tickers, same start baseline
  const mkt = meanOverTickers(row.prices, game.meta.start_prices); // index level
```

```
  const vsMarket = absRet - (mkt - 1) * 100;     // vs. Market Return %
  render({ cash, unrealised, total, absRet, sharpe, vsMarket });
}
```

Card	Formula (plain)
Available Cash	start cash − Σ buys + Σ sells
Unrealised Gain	Σ over holdings of shares × (current price − average cost)
Total Portfolio	cash + Σ (shares × current price)
Absolute Return %	(total − starting capital) / starting capital × 100
Sharpe Ratio	mean(portfolio tick-returns) / std(portfolio tick-returns)
vs. Market Return %	portfolio return % − equal-weight market return %

6.6 Research tab — the universe table

Render game.universe as the table. Each row's ResDocPrev link opens that entry's research text in a panel or modal. The Filter box just filters the rows by ticker/company substring.

```
game.universe.forEach(u => addRow(u.ticker, u.company, currentPrice(u.ticker),
                                  change(u.ticker), u.type,
                                  openDocButton(u.research)));
```

6.7 Optional: keep every player in sync

Because each browser runs its own 10-second timer, players can drift a few seconds apart. For a competitive event, drive the tick from the server (a websocket that broadcasts 'now on tick t') instead of a local timer, so everyone reveals the same row at the same moment. The playback logic is otherwise identical.

Part 7 — Tuning the feel (owner)

The model's behaviour is decided by a handful of dials. The reliable way to tune is to run a few hundred seeds and look at the distribution of outcomes (as in the fan chart), then nudge one dial at a time.

Dial	Turn it up to…	Watch out for
GAIN_SCALE / event gain	make news matter more vs. random noise	too high = the chart is 'obvious', no skill
SCORE_SPREAD (σ_s)	make outcomes more surprising / two-sided	too high = news stops teaching a direction
IDIO_MULT (σ_i)	add private, unpredictable wiggle	too high = signal drowns in noise
κ (reversion)	make overshoots retrace faster	too high = shocks feel weightless
ϕ (perm_frac)	let news permanently change fair value	too high = 'short forever' returns
event tau (minutes)	make a story linger longer	longer than the session = never retraces
drift μ	reward being net-long a rising tape	should stay small vs. news moves

The two ratios that decide 'fun vs. random'

(1) news size vs. idiosyncratic noise — too low and news is invisible; too high and the game is 'read the headline, done'. (2) drift vs. volatility — sets how much picking the right direction pays over a session. Find the sweet spot by simulating a few hundred runs and eyeballing the spread of final returns.

Appendix A — the 10 factors

CrudeOil · RatesRupee · GlobalTech · DomPolicy · Metals · ConsDemand · GlobalRisk · Credit · EnergyTx · SupplyChain. Kept deliberately different so each headline maps cleanly to one theme.

Appendix B — the 10 stocks and their main exposures

Ticker	Company	Start ₹	Main exposures (sign)
TCS	Tata Consultancy	3850	GlobalTech +, GlobalRisk +, Rates −
INFY	Infosys	1520	GlobalTech +, GlobalRisk +, Rates −
HDFCBANK	HDFC Bank	1650	Rates +, Credit +, DomPolicy +
ICICIBANK	ICICI Bank	1120	Rates +, Credit +, DomPolicy +
RELIANCE	Reliance Industries	2900	CrudeOil +, DomPolicy +, EnergyTx +
ONGC	Oil & Natural Gas	270	CrudeOil +, EnergyTx −
TATAMOTORS	Tata Motors	950	CrudeOil −, Metals −, Rates +, ConsDemand +
TATASTEEL	Tata Steel	140	Metals +, GlobalRisk +, SupplyChain +
HINDUNILVR	Hindustan Unilever	2400	ConsDemand +, CrudeOil −
SUNPHARMA	Sun Pharmaceutical	1500	DomPolicy +, SupplyChain −

— End of handover. The engine (market_engine.py), a sample game_data.json, and all figures accompany this document in the same folder.